FPT UNIVERSITY

EMBEDDED LINUX SYSTEMS

BÁO CÁO ASSIGNMENT 01

How to Write a Device Driver

Giao diện ioctl và misc device trên ARMv7

Thông tin	Nội dung
Sinh viên	Phạm Trần Gia Long
Mã số sinh viên	SE201095
Nền tảng	QEMU vexpress-a9, ARMv7, Linux 5.15.0
Driver	asgn1_driver và counter_driver
Ngày kiểm thử	21 tháng 10 năm 2026

Phạm vi báo cáo

Báo cáo phân tích thiết kế hai character driver: driver bộ đệm có bốn lệnh ioctl và driver bộ đếm đăng ký qua miscdevice. Các kết quả thực nghiệm gồm đọc ghi dữ liệu, đổi chế độ, reset, thống kê, xử lý đầu vào sai và thu hồi tài nguyên.

Trong phiên kiểm thử sau sửa, cả test_counter.sh và test_asgn1.sh đều kết thúc PASSED, với mã thoát 0. Bản mã cuối đã đưa dòng log đọc open_count vào mutex và bổ sung mô tả cho các hàm của hai driver. Mục 15 trình bày lỗi đã sửa và giới hạn của kết luận kiểm thử.

1 Môi trường và cơ sở kiểm chứng

Thành phần	Giá trị thực tế
Máy phát triển	Ubuntu trên ASUS TUF Gaming F15 FX507ZC4
Compiler	Arm GNU Toolchain 13.2.rel1, GCC 13.2.1
Kernel đích	Linux 5.15.0 SMP, ARMv7
Máy ảo	vexpress-a9, Cortex-A9, 2 CPU, RAM 512 MB
Root filesystem	BusyBox và initramfs dạng cpio gzip
Driver Task 1	/dev/asgn1, major 241, minor 0
Driver Task 2	/dev/counter, major 10, minor 125 trong phiên đo
Thư mục làm việc	~/assignment01/task1_ioctl và task2_counter

Minor của miscdevice được cấp động nên giá trị 125 chỉ mô tả phiên chạy đã ghi nhận; không được xem đây là giá trị cố định trong mã nguồn. Các thông tin compiler và kernel được đối chiếu từ dòng Linux version trong dmesg_full.txt.

Tổ chức nội dung

Mục 2 mô tả kiến trúc chung. Mục 3 đến 6 phân tích ioctl và kết quả Task 1. Mục 7 đến 10 trình bày counter và các phép thử. Mục 11 đến 14 phân tích reference count, so sánh driver, MMU, MTD và EOF. Mục 15 đến 17 tổng hợp debug, cách tái lập và tài liệu tham khảo.

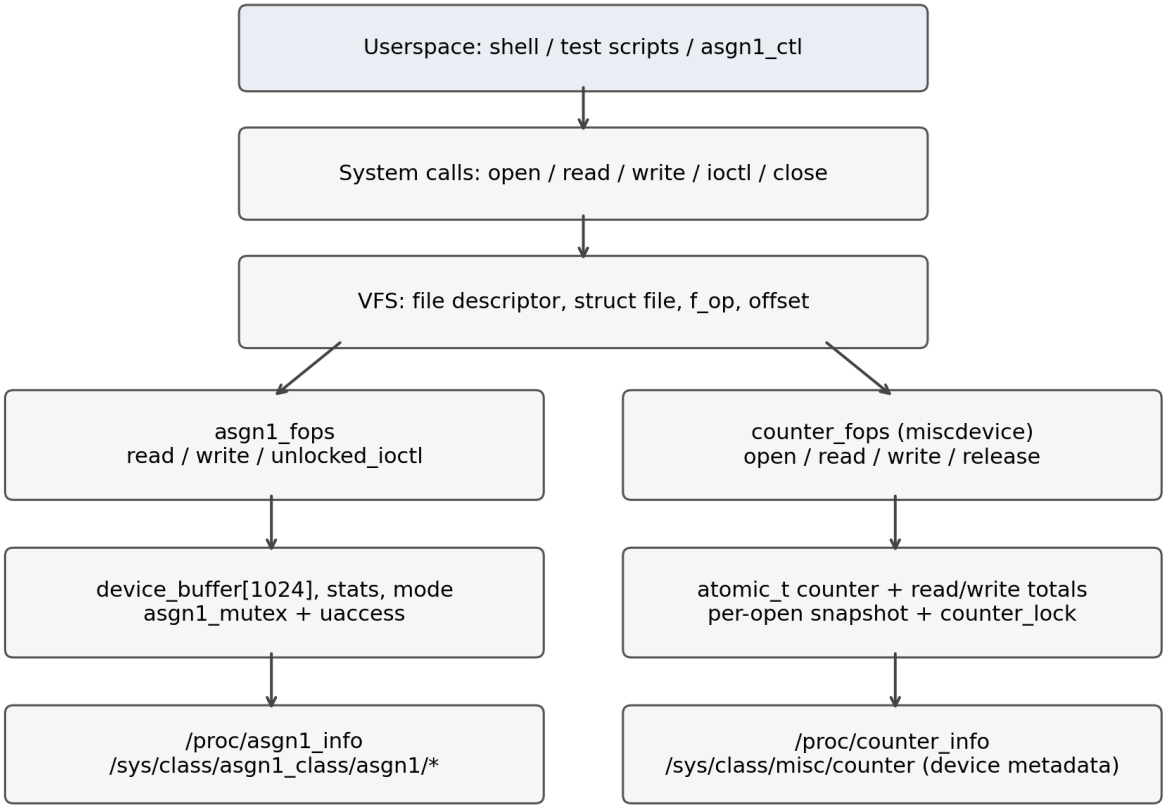
Đối chiếu bằng chứng

Các ảnh chức năng được chụp ở phiên chạy thủ công; log nộp bài được lấy từ phiên sau sửa chạy cả hai script, lưu trong postfix_session_raw.txt. Giá trị nghiệp vụ nhất quán giữa các phiên. Phép thử giữ FD 3 và gỡ module nằm trong task2_session_raw.txt, không nằm trong test_counter.sh.

Log đầy đủ được lưu riêng trong thư mục logs của bài nộp. Báo cáo đưa các bảng đối chiếu và ảnh có thể đọc trực tiếp; các nhánh kiểm thử chưa chạy được nêu riêng, không suy từ việc biên dịch thành công.

2 Kiến trúc chung của hai driver

Ứng dụng userspace thao tác qua file descriptor. System call chuyển yêu cầu vào kernel; VFS và lớp character device điều phối đến bảng file_operations của thiết bị. Hai driver có dữ liệu độc lập và không gọi trực tiếp lẫn nhau. [S1, S3]



Hình 1. Sơ đồ phân tích chung dựng theo hai file nguồn trong gói kiểm thử

Đường điều khiển ioctl

asgn1_ctl mở /dev/asgn1 rồi gọi ioctl(fd, cmd, arg). Kernel chuyển yêu cầu tới asgn1_fops.unlocked_ioctl, tức asgn1_ioctl. Với SET_MODE, copy_from_user lấy int từ userspace trước khi đổi mode. Với GET_STATS và GET_VERSION, copy_to_user chuyển dữ liệu trở lại. RESET_BUFFER chỉ sửa trạng thái nội bộ, không cần arg. [S1, S2]

Procfs và sysfs là các đường quan sát song song: callback show đọc trạng thái driver rồi định dạng chuỗi. Đọc /proc/counter_info không đi qua counter_read nên không tăng counter. Sysfs của counter trong sơ đồ là metadata thiết bị do miscdevice tạo, không phải thuộc tính counter tùy biến.

3 Giao thức ioctl của Task 1

asgn1_ioctl.h là hợp đồng chung giữa module và chương trình điều khiển. Magic là ký tự G, mã ASCII 0x47. Struct asgn1_stats có năm trường int; trong ABI ARM EABI đã dùng, tổng kích thước là 20 byte. [S1]

Lệnh	Hướng	NR	SIZE	Mã trong log
RESET_BUFFER	_IO	0	0	0x00004700
GET_STATS	_IOR	1	20	0x80144701
SET_MODE	_IOW	2	4	0x40044702
GET_VERSION	_IOR	3	64	0x80404703

Hướng đọc ghi trong tên macro được hiểu từ userspace: _IOR nhận dữ liệu từ kernel; _IOW gửi dữ liệu vào kernel. Magic giúp phân biệt nhóm lệnh, không phải cơ chế xác thực hay bảo mật. Switch trên toàn bộ cmd chỉ nhận đúng bốn mã trong header. [S2]

Giải mã ví dụ thực tế

```
#define ASGN1_GET_STATS \
    _IOR(ASGN1_MAGIC, 1, struct asgn1_stats)
```

Với bố trí bit của ABI đang dùng, NR nằm ở 8 bit thấp, TYPE ở 8 bit tiếp theo, SIZE ở 14 bit kế và DIR ở 2 bit cao. Vì vậy GET_STATS có NR = 1, TYPE = 0x47, SIZE = 20 và DIR = 2. Mã tính được là (2 << 30) | (20 << 16) | (0x47 << 8) | 1 = 0x80144701.

_IOC_NR(ASGN1_SET_MODE) = 2 và _IOC_SIZE(ASGN1_SET_MODE) = sizeof(int) = 4. GET_VERSION dùng char[64] nên SIZE = 64; không truyền sizeof(char[64]) làm tham số kiểu dữ liệu cho _IOR.

Mã trả về và dữ liệu khởi tạo

Các lệnh hợp lệ trả 0 khi thành công. Driver trả -ENOTTY cho cmd lạ, -EINVAL cho mode khác 0 hoặc 1, và -EFAULT nếu sao chép qua biên userspace thất bại. Mảng version được khởi tạo đủ 64 byte trước khi copy để không xuất phần stack chưa khởi tạo. [S1]

4 Bộ đệm và đồng bộ trong Task 1

Đọc ghi và tính nhất quán

asgn1_write cấp phát temp bằng kmalloc trước khi lấy mutex, kiểm tra kích thước theo mode rồi mới nhận dữ liệu userspace. Nội dung cũ chỉ bị thay thế sau khi copy_from_user hoàn tất. Nếu có lỗi, nhánh out nhả mutex và kfree(temp); buffer cùng các bộ đếm chưa được cập nhật. [S1]

BUFFER_SIZE là 1024 byte. Echo nhận tối đa 1023 byte, dành một byte NUL; kernel mode dành thêm 8 byte cho [KERNEL] nên nhận tối đa 1015 byte. last_write_size là số byte ứng dụng ghi, còn buffer_len tính cả tiền tố. Vì vậy XYZ có last_write_size = 3 nhưng buffer_len = 11 trong mode 0.

asgn1_read giữ mutex khi kiểm tra buffer_len, copy_to_user, tăng offset và read_count. Offset âm trả -EINVAL; yêu cầu 0 byte trả 0. Nếu offset đạt buffer_len, hàm trả 0 để cat kết thúc. RESET không tăng read_count khi lần đọc tiếp theo chỉ gặp EOF.

Vai trò của mutex

Một mutex chung bảo vệ quan hệ giữa nội dung buffer, độ dài, mode và thống kê. Nếu chỉ khóa từng biến riêng, GET_STATS có thể lấy buffer_len mới cùng last_write_size cũ. Giữ khóa khi tạo snapshot giúp các trường mô tả cùng một trạng thái.

Các thao tác uaccess có thể ngủ khi xử lý page fault, nên không đặt chúng trong vùng spinlock. Mutex phù hợp với callback chạy trong process context. Trong bản cuối, phần cập nhật và các đường đọc trạng thái đều dùng mutex. Dòng pr_info trong asgn1_open cũng nằm trước mutex_unlock để tránh đọc open_count ngoài vùng bảo vệ. [S1, S4]

Đăng ký và thu hồi

Init lần lượt đăng ký major 241, class, device, nhóm thuộc tính sysfs và procfs. Exit gỡ procfs, thuộc tính, device, class và major. Các lỗi class_create hoặc device_create có nhánh thu hồi tài nguyên trước đó. Khi tạo procfs hay nhóm sysfs thất bại, bản hiện tại chỉ cảnh báo rồi tiếp tục; có thể đổi sang rollback nếu coi hai giao diện này là bắt buộc. Chưa thực hiện fault injection cho init.

5 Kiểm thử version và chế độ ghi

Version trả về đúng asgn1_driver v1.0 - SE201095. Echo mặc định giữ ABC; sau SET_MODE 0, dữ liệu XYZ được lưu thành [KERNEL]XYZ. [S1]

```
Expected: [ABC]
Actual:   [ABC]
asgn1_driver: device opened (count=4)
asgn1_driver: ioctl cmd=0x80144701
asgn1_driver: GET_STATS result=0
asgn1_driver: device closed
open_count=4
write_count=1
read_count=1
buffer_len=3
last_write_size=3
```

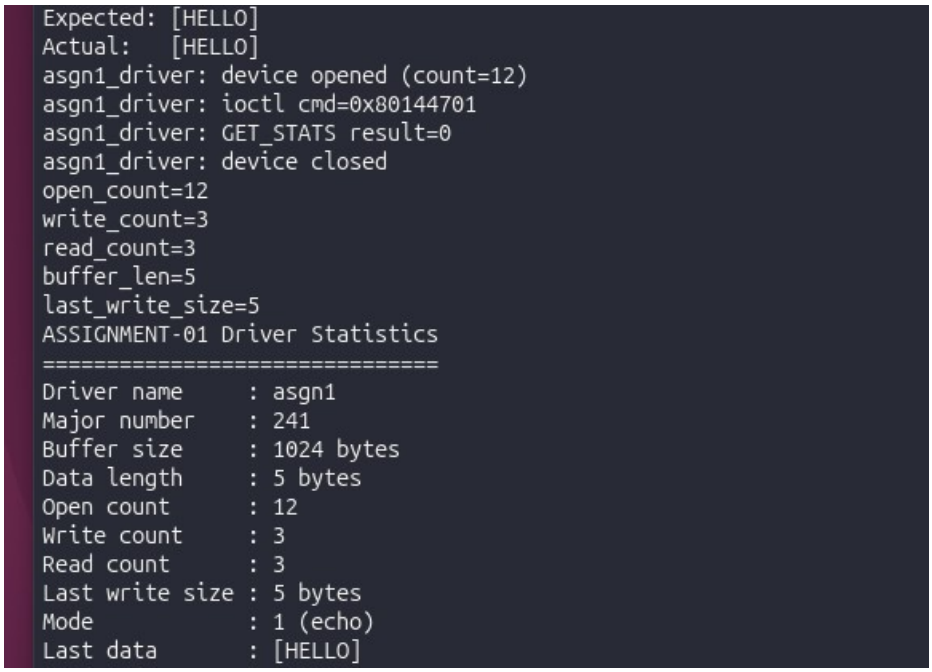
Hình 2. Trích phần đọc lại ABC và GET_STATS ở echo mặc định

```
[4] Kernel mode
asgn1_driver: device opened (count=5)
asgn1_driver: ioctl cmd=0x40044702
asgn1_driver: SET_MODE mode=0
SET_MODE OK: 0 (kernel)
asgn1_driver: device closed
asgn1_driver: device opened (count=6)
asgn1_driver: write 3 bytes, mode=0, buffer_len=11, write_count=2
asgn1_driver: device closed
asgn1_driver: device opened (count=7)
asgn1_driver: read 11 bytes, read_count=2
asgn1_driver: device closed
Expected: [[KERNEL]XYZ]
Actual:   [[KERNEL]XYZ]
asgn1_driver: device opened (count=8)
asgn1_driver: ioctl cmd=0x80144701
asgn1_driver: GET_STATS result=0
asgn1_driver: device closed
open_count=8
write_count=2
read_count=2
buffer_len=11
last_write_size=3
```

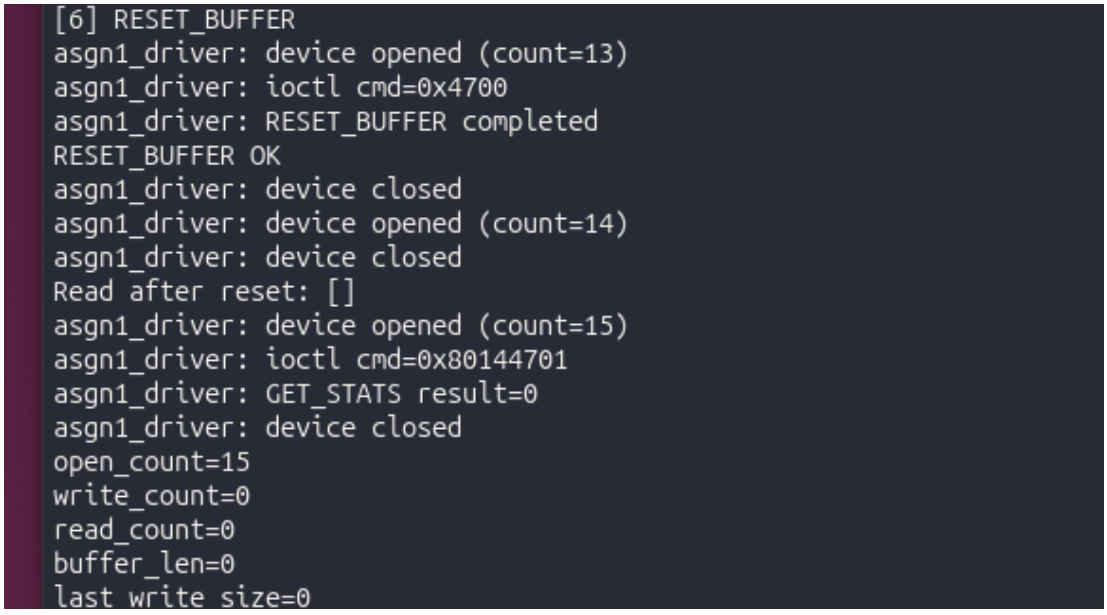
Hình 3. Mode 0 thêm tiền tố và cập nhật buffer_len thành 11

Hai mốc có write_count/read_count lần lượt 1/1 và 2/2. Mỗi lần gọi asgn1_ctl đều open thiết bị nên open_count cũng tăng; đây là tổng số lần mở kể từ khi load, không phải số file đang mở.

6 Kiểm thử echo và reset của Task 1



Hình 4. HELLO và procfs thống kê tương ứng sau khi chuyển lại echo



Hình 5. RESET_BUFFER xóa dữ liệu và đưa các bộ đếm đọc ghi về 0

Sau HELLO: write_count = 3, read_count = 3, buffer_len = 5 và last_write_size = 5. Sau RESET: bốn trường này đều 0; open_count = 15, mode vẫn 1. Reset không có yêu cầu đặt lại open_count hoặc mode. [S1]

test_asgn1_output.txt ghi đủ GET_VERSION, GET_STATS, SET_MODE cho cả hai chế độ, RESET_BUFFER và rmmod. Phiên cuối kết thúc ASGN1 TEST PASSED với asgn1_script_exit = 0.

7 Thiết kế counter qua miscdevice

counter_driver là module riêng dùng struct miscdevice, MISC_DYNAMIC_MINOR và misc_register. Framework cung cấp major 10; tên counter tạo node /dev/counter. Khi proc_create thất bại sau misc_register, init gọi misc_deregister rồi trả -ENOMEM. Exit xóa procfs trước khi hủy miscdevice. [S1, S3]

Dữ liệu toàn cục và dữ liệu mỗi lần mở

Trạng thái	Ý nghĩa
atomic_t counter	Giá trị chung, khởi tạo 0
atomic_t total_reads	Số callback read trả số byte dương
atomic_t total_writes	Số write hợp lệ, kể cả đặt số khác 0
counter_file_state	Chuỗi text, length, started thuộc mỗi open
counter_lock	Khóa giao dịch đọc tăng, ghi đặt và snapshot procfs

counter_open dùng kzalloc tạo trạng thái riêng rồi gán file->private_data. counter_release kfree vùng này khi struct file hết tham chiếu. Vì driver tự định nghĩa open, giá trị private_data mặc định của misc framework được thay bằng trạng thái của từng lần mở.

Chu trình đọc

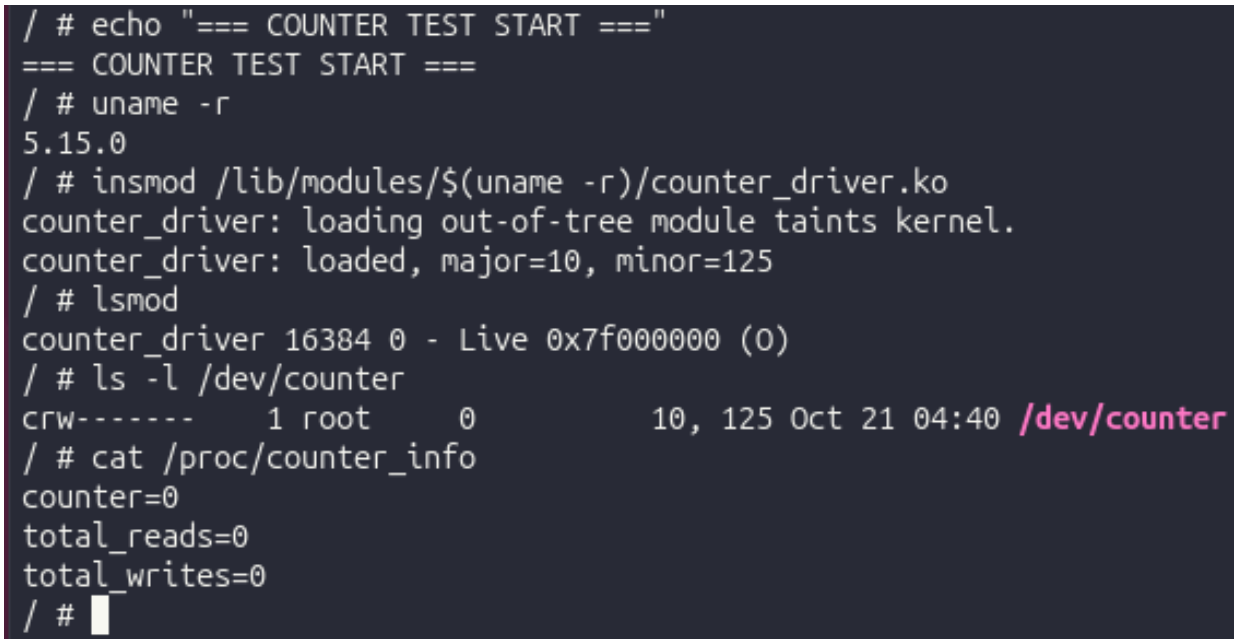
Ở lần read thành công đầu tiên, driver lấy giá trị counter, định dạng chuỗi thập phân có newline, chuyển dữ liệu bằng simple_read_from_buffer rồi atomic_inc. started giữ snapshot ổn định cho các lần đọc phần còn lại; ppos được helper cập nhật. Khi hết chuỗi, read trả 0.

Nếu ứng dụng đọc từng byte, counter chỉ tăng một lần cho bản ghi của open đó; total_reads tăng cho mỗi read trả byte dương. Với cat trong thử nghiệm, mỗi bản ghi chỉ cần một read dương nên total_reads trùng số lần tăng. Trường hợp đọc từng byte chưa được kiểm thử thực nghiệm.

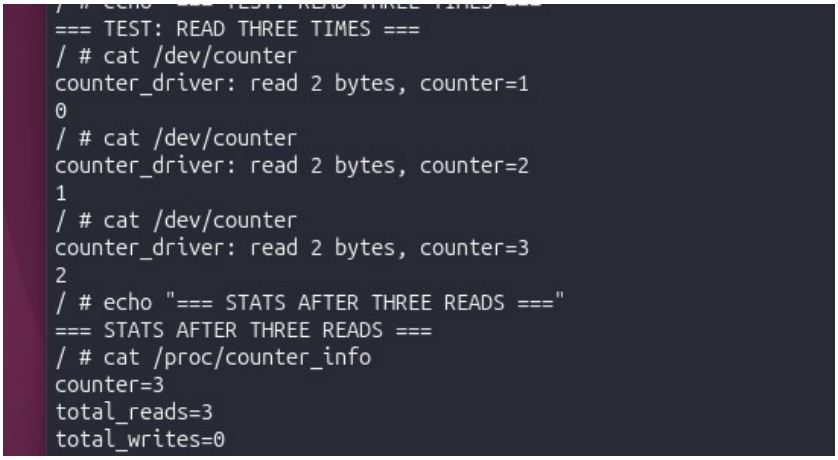
Chu trình ghi

Driver nhận tối đa 31 byte vào text[32], từ chối NUL nhúng, thêm NUL cuối và gọi kstrtoint cơ số 10. Số âm hoặc parse lỗi trả -EINVAL; dữ liệu sai không tăng total_writes. Giá trị hợp lệ được atomic_set dưới mutex. Trường hợp counter = INT_MAX được bảo vệ bằng -EOVERFLOW khi bắt đầu bản ghi đọc, thay vì tăng sang số âm. [S1]

8 Kiểm thử nạp counter và tự tăng



Hình 6. Nạp counter_driver, major 10 minor 125 và trạng thái ban đầu

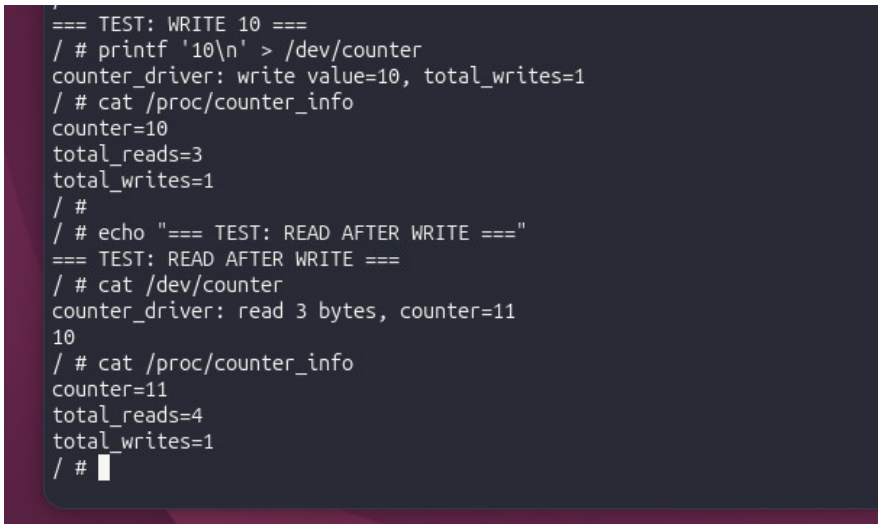


Hình 7. Ba lần cat trả 0, 1, 2 và thống kê sau đọc

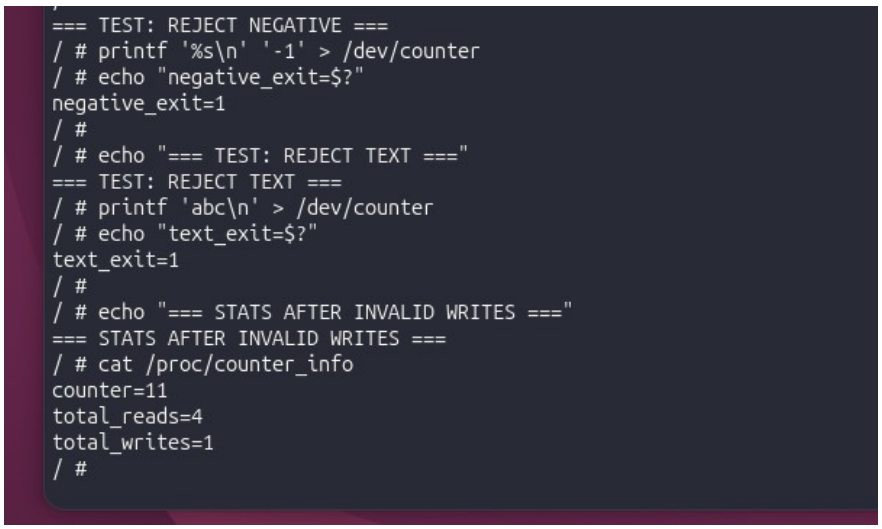
Ban đầu counter/total_reads/total_writes đều 0. Sau ba lần đọc, counter = 3, total_reads = 3 và total_writes = 0. Mỗi lệnh cat trở lại shell, là bằng chứng đường đọc thông thường kết thúc được tại EOF. [S1]

Dòng loading out-of-tree module taints kernel cho biết module được build ngoài cây kernel. Chỉ riêng thông báo này không chứng minh module lỗi; trong phiên chạy, insmod tiếp tục thành công và các phép thử đọc ghi hoạt động.

9 Kiểm thử ghi và từ chối dữ liệu sai



Hình 8. Ghi 10, đọc trả 10 rồi counter tăng thành 11

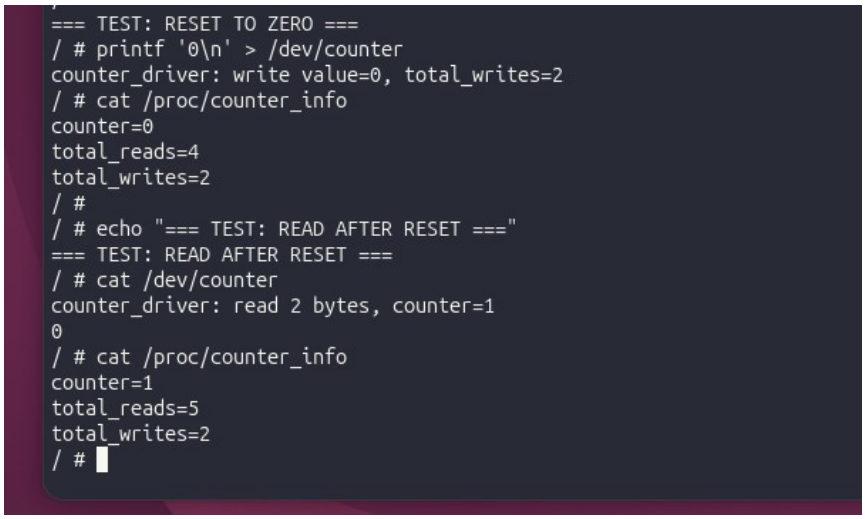


Hình 9. Ghi số âm và chuỗi chữ đều thất bại, thống kê không đổi

Sau ghi 10: counter = 10, total_reads = 3, total_writes = 1. Đọc lại trả 10 và chuyển trạng thái sang 11/4/1. Hai lần ghi -1 và abc có exit status 1, giữ nguyên trạng thái 11/4/1. [S1]

Shell không in tên errno cho hai lần printf lỗi. Vì vậy bằng chứng runtime chỉ xác nhận thao tác bị từ chối; -EINVAL được xác định bằng đối chiếu nhánh xử lý trong counter_write, chưa đo errno bằng một chương trình C riêng.

10 Kiểm thử reset và script tự động



Hình 10. Ghi 0 reset counter, tổng đọc ghi không bị xóa



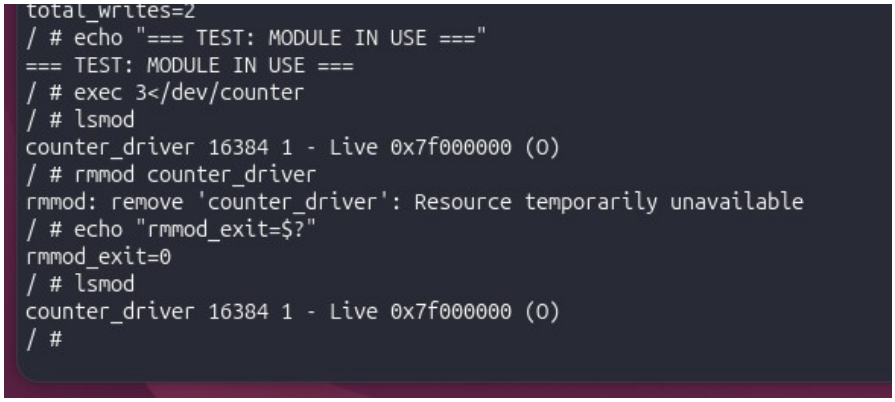
Hình 11. Phần cuối script counter với PASSED và counter_script_exit bằng 0

Mốc procfs	counter	reads	writes
Ban đầu	0	0	0
Sau ba lần đọc	3	3	0
Sau ghi 10	10	3	1
Sau đọc 10	11	4	1
Sau đầu vào sai	11	4	1
Sau ghi 0	0	4	2
Sau đọc 0	1	5	2

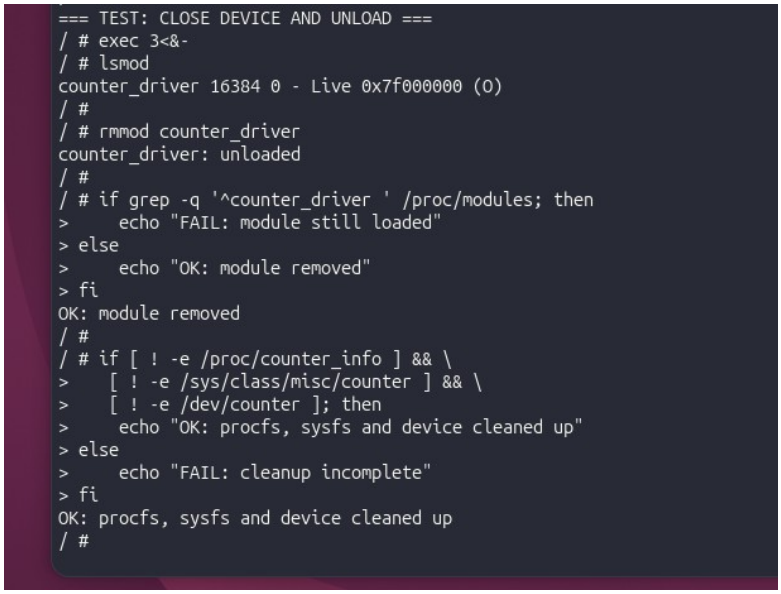
test_counter.sh tự so sánh dữ liệu và thống kê, dừng khi sai. Cuối script kiểm tra module, procfs, sysfs và node đã được gỡ. Các mốc trên đáp ứng việc quan sát procfs trước write, sau write và sau reset. [S1]

11 Reference count và gỡ module

Câu 1. Với file_operations.owner = THIS_MODULE, lớp mở thiết bị giữ tham chiếu module; khi struct file được giải phóng ở lần đóng cuối, tham chiếu tương ứng được thả. Các lời gọi try_module_get/module_put và quan hệ phụ thuộc module cũng tham gia quản lý vòng đời. Refcount không phải tổng số lần open và không nhất thiết bằng số file descriptor: dup có thể chia sẻ cùng struct file.



Hình 12. FD 3 giữ thiết bị mở, refcount 1 và rmmod không gỡ được module



Hình 13. Đóng FD 3 đưa refcount về 0, gỡ module và cleanup thành công

Trong phép đo, rmmod báo Resource temporarily unavailable nhưng shell ghi rmmod_exit = 0. Chưa xác định nguyên nhân khác biệt của mã thoát. Việc module còn trong /proc/modules với refcount 1 xác nhận gỡ thất bại; sau exec 3<&-, module được gỡ thực sự. [S1]

Có thể yêu cầu rmmod -f nếu công cụ hỗ trợ và kernel bật CONFIG_MODULE_FORCE_UNLOAD. Force bỏ qua kiểm tra sử dụng trong trường hợp được hỗ trợ, có nguy cơ để callback trỏ vào mã đã thu hồi. Thử nghiệm này đóng FD rồi gỡ bình thường; không thử force unload. [S5]

12 So sánh ba loại device driver

Tiêu chí	Character	Block	Network
Giao diện	file_operations qua VFS	block_device_operations và hàng đợi block I/O	net_device_ops qua network stack
Đơn vị	Byte hoặc bản ghi do driver quy định	Sector và block I/O	Packet hoặc frame
Buffering	Driver tự chọn buffer/ring; không có page cache file thường mặc định	Page cache khi buffered I/O, bio/request, có thể dùng direct I/O	sk_buff, TX/RX queues, DMA rings
Thiết bị nhúng	UART, watchdog, GPIO điều khiển	eMMC, thẻ SD, NVMe	Ethernet MAC, Wi-Fi
Điểm truy cập	open/read/write/ioctl trên node	Filesystem hoặc raw block device	Socket API, netlink; không phải open eth0 như file

block_device_operations không tự thay thế toàn bộ đường truyền dữ liệu; dữ liệu thực đi qua block layer và cơ chế hàng đợi. Với network device, net_device_ops không phải bảng file_operations của một node /dev; ứng dụng gửi nhận qua socket và network stack. [S6]

Các quyết định thiết kế cụ thể

Một thanh ghi điều khiển LED hoặc bộ đếm chẩn đoán chỉ cần vài byte trạng thái nên chọn character device, có thể dùng miscdevice để giảm mã đăng ký. Không cần mô hình sector hay bộ lập lịch lưu trữ cho dữ liệu này.

Thiết bị ghi dữ liệu cảm biến lên thẻ SD, cần filesystem và truy cập vị trí, nên dùng block subsystem do thẻ cung cấp. Đối với camera truyền ảnh qua Ethernet, chức năng truyền mạng sử dụng network stack để tận dụng socket, định tuyến và xử lý packet.

register_chrdev và misc_register

Task 1 dùng register_chrdev để kế thừa cấu trúc LAB02 và thể hiện rõ major, class, device cùng sysfs riêng. Cách này chủ động nhưng có nhiều bước cleanup và nguy cơ trùng major cố định. Task 2 chỉ có một chức năng nên misc_register với minor động ngắn gọn hơn; major được dùng chung là 10. Cả hai vẫn là character device. [S1, S3]

13 Atomic và xử lý EOF

Race condition cụ thể

Giả sử counter là int thường có giá trị 7. CPU A đọc 7, CPU B cũng đọc 7; A ghi 8 và B ghi 8. Hai lần tăng chỉ tạo kết quả 8 thay vì 9. atomic_inc thực hiện cập nhật không bị mất lần tăng theo kiểu read modify write này. atomic_read và atomic_set là các thao tác truy cập tương ứng, không làm cả một chuỗi nhiều thao tác trở thành nguyên tử. [S7]

Ngay cả khi từng thao tác atomic, chuỗi atomic_read → copy_to_user → atomic_inc vẫn có thể cho hai người đọc cùng giá trị nếu không có khóa bao quanh. counter_lock tuần tự hóa cả giao dịch, bao gồm write đặt lại giá trị; nhờ vậy mỗi bản ghi đọc thành công nhận một snapshot xác định. Mutex cũng giữ ba thống kê procfs nhất quán. [S1]

Câu 4 về offset và cat

cat ở userspace lặp gọi read, in các byte nhận được rồi gọi tiếp. Mỗi system call đi qua VFS đến callback read; khi callback trả 0, ứng dụng coi là EOF và dừng. VFS không tự tạo vòng lặp in ra terminal thay cat. Read trả lỗi cũng có thể làm cat dừng với trạng thái lỗi.

Nếu callback luôn định dạng số mới và trả số byte dương, mỗi lần cat đọc sẽ lại tăng counter, khiến cat tiếp tục in không kết thúc trong điều kiện bình thường. Vì vậy offset hoặc trạng thái mỗi lần mở phải phân biệt bản ghi đang đọc với yêu cầu tạo bản ghi mới.

```
ret = simple_read_from_buffer(buffer, count, ppos,
                              state->text, state->length);
if (ret > 0) {
    if (!state->started) {
        atomic_inc(&counter);
        state->started = true;
    }
    atomic_inc(&total_reads);
}
```

Đoạn trích từ counter_read thực tế. Khi ppos đạt state->length, helper trả 0, không tăng counter hay total_reads. Mở thiết bị lần mới tạo state mới nên lần cat kế tiếp nhận giá trị mới. Các read phần nhỏ trong cùng open dùng chung chuỗi snapshot. [S1]

14 Truy cập userspace và lưu trữ flash

Câu 2 về copy_to_user và MMU

GET_STATS tạo struct trong kernel rồi dùng copy_to_user đến địa chỉ userspace. Con trỏ arg không được tin cậy: địa chỉ có thể chưa ánh xạ, không có quyền ghi hoặc thay đổi trong lúc xử lý. memcpy chỉ sao chép byte, không cung cấp hợp đồng kiểm tra và khôi phục lỗi của uaccess.

Ở cấp MMU, địa chỉ ảo được dịch qua page table và kiểm tra quyền trang. Ghi vào trang không hiện diện hoặc bị cấm có thể gây data abort/page fault; một địa chỉ user hợp lệ cũng có thể truy cập được trong một số cấu hình. Vì vậy không đúng khi nói mọi ghi trực tiếp đều chắc chắn bị MMU chặn. API uaccess cung cấp đường truy cập phù hợp kiến trúc và cơ chế xử lý lỗi thay vì phụ thuộc vào một phép dereference tùy ý.

copy_to_user trả số byte chưa sao chép được; driver kiểm tra khác 0 và trả -EFAULT. __user là chú thích hỗ trợ phân tích tĩnh, không tự kiểm tra địa chỉ lúc chạy. Bản ghi thống kê được khởi tạo trước khi copy để hạn chế lộ dữ liệu kernel. [S1, S4]

Câu 3 về MTD và block subsystem

MTD biểu diễn flash thô với thao tác đọc, lập trình và xóa eraseblock. Phần mềm phải quan tâm quy tắc xóa trước ghi lại, độ mòn và đặc tính bad block của NAND. Block subsystem cung cấp địa chỉ sector logic có thể đọc ghi; nhiều thiết bị managed flash đã xử lý ánh xạ và quản lý flash bên trong controller. [S8]

Ví dụ, NAND thô hàn trực tiếp trên bo mạch nhúng cần MTD để quản lý eraseblock, bad block và ECC tùy controller; có thể dùng UBI và UBIFS ở phía trên. SPI NOR dùng lưu bootloader cũng là ví dụ MTD. Ngược lại, eMMC và thẻ SD thường xuất block device vì controller của chúng che phần lớn chi tiết flash thô.

UBI làm quản lý volume và wear leveling phía trên MTD; UBIFS hoạt động trên UBI volume. Việc một bộ nhớ có bản chất flash không tự động có nghĩa phải dùng MTD: lựa chọn phụ thuộc giao diện mà phần cứng thực sự xuất cho hệ điều hành. [S8]

15 Debug và kết quả rà soát

Lỗi thiếu dấu chấm phẩy trước nhãn out

Trong lúc sửa asgn1_read, compiler báo expected semicolon before out và label out used but not defined. Lỗi gốc là lời gọi pr_info ngay trước out thiếu dấu chấm phẩy; parser không hoàn tất câu lệnh nên báo tiếp lỗi nhãn. Kiểm tra vài dòng trước vị trí báo lỗi giúp xác định nguyên nhân. [S1]



Hình 14. Ảnh mã lúc debug cho thấy pr_info chưa kết thúc bằng dấu chấm phẩy

Cách sửa là kết thúc pr_info bằng);, giữ nhãn out: ở đầu nhánh nhả mutex rồi make lại. File nguồn cuối trong gói có dấu chấm phẩy và log chạy xác nhận đường đọc hoạt động. Một lỗi nhập liệu khác là ký tự w trước /* ở dòng đầu đã được loại bỏ trước khi build thành công.

Kết quả sửa và kiểm thử lại

Bản trước có open_count++ dưới mutex nhưng pr_info đọc biến sau unlock. Bản cuối đã chuyển pr_info lên trước mutex_unlock, giữ việc tăng và ghi log trong cùng vùng bảo vệ. Chú thích cũ cho rằng read không khóa cũng đã được sửa để khớp mã. Thay đổi này không đổi giao thức ioctl hay ý nghĩa bộ đếm.

Cả 12 hàm của asgn1_driver và 8 hàm của counter_driver đã có comment mô tả mục đích, tham số và giá trị trả về. Hai module được build lại trên Ubuntu, chép vào rootfs và chạy lại trên QEMU. Log postfix_session_raw.txt ghi cả hai script PASSED và mã thoát 0; gói cuối có các binary ARM và log sau sửa.

Giới hạn kiểm thử

Chưa thử tải đồng thời nhiều tiến trình, đọc từng byte, user pointer không hợp lệ, biên kích thước và INT_MAX, hoặc ép lỗi cấp phát/init. Các nhánh bảo vệ tương ứng mới được rà soát từ source. Trong log có ioctl 0x5401 ngoài bốn mã riêng; đây là mã TCGETS của ABI đang dùng. Khả năng shell thăm dò terminal là suy luận, chưa xác minh bằng trace.

16 Tái lập và cấu trúc bài nộp

Build và chạy trên đúng kiến trúc

Hai Makefile gọi Kbuild trong ~/embedded_lab1/kernel/linux-5.15 với ARCH=arm và CROSS_COMPILE=arm-none-linux-gnueabihf-. Module phải khớp kernel đích về version và cấu hình. asgn1_ctl được liên kết static để chạy trong rootfs BusyBox mà không cần bổ sung dynamic loader. [S1]

```
cd ~/assignment01/task1_ioctl
make
arm-none-linux-gnueabihf-gcc -static -O2 -Wall -Wextra \
    asgn1_ctl.c -o asgn1_ctl
cd ~/assignment01/task2_counter
make
```

Chép hai .ko vào /lib/modules/5.15.0 trong rootfs; chép helper và hai script vào /usr/bin, đặt quyền thực thi. Sau khi đóng gói lại initramfs, khởi động QEMU vexpress-a9 với zImage, DTB vexpress-v2p-ca9 và console ttyAMA0. Chạy hai script khi các module tương ứng chưa được nạp.

Gói nộp cuối

```
SE201095_PhamTranGiaLong_Asgn01/
  task1_ioctl/
    asgn1_driver.c  asgn1_ioctl.h  Makefile
    asgn1_driver.ko  test_asgn1.sh
    asgn1_ctl.c  asgn1_ctl
  task2_misc/
    counter_driver.c  Makefile
    counter_driver.ko  test_counter.sh
  logs/
    test_asgn1_output.txt
    test_counter_output.txt
    dmesg_full.txt
  bao_cao/
    SE201095_PhamTranGiaLong_Asgn01_BaoCao.pdf
```

Thư mục làm việc Task 2 hiện là task2_counter; khi tạo cây nộp, chép nội dung sang task2_misc theo đề. Gói cuối có source sau sửa, hai file .ko ARMv7, helper ARM liên kết static và log kiểm thử sau sửa. Phiên QEMU kết thúc ngày 21/10/2026 lúc 13:01:26 giờ Việt Nam. Các binary được giữ nguyên từ gói FinalFiles gửi sau khi build và kiểm thử lại.

Hai log chức năng lưu nguyên trình tự từ nạp đến gỡ; dmesg_full.txt chứa log kernel sau khi chạy cả hai test. Giữ raw session để truy nguyên các dòng đã tách. Không dùng kết quả phiên cũ để chứng minh một binary đã thay đổi.

17 Tài liệu tham khảo và kết luận kiểm thử

Kết quả đã xác nhận: Task 1 thực hiện đủ bốn ioctl và đúng hai chế độ ghi; RESET làm rỗng buffer và xóa các bộ đếm đọc ghi. Task 2 đọc tăng, ghi đặt số, reset và từ chối đầu vào sai; procfs phản ánh đúng các mốc đo. Gỡ module sau khi đóng thiết bị thu hồi node và giao diện thống kê. [S1]

Các kết quả trên có phạm vi là những ca đã chạy trong log. Truy cập open_count ngoài mutex đã được sửa và comment hàm đã được bổ sung trước khi build, kiểm thử lại và chốt binary. Các ca biên, fault injection và tải đồng thời chưa chạy vẫn là giới hạn được nêu tại mục 15.

Nguồn thực nghiệm

[S1] Phạm Trần Gia Long. Gói SE201095_Asgn01_FinalFiles.tar.gz, ngày 21/10/2026: hai driver sau sửa, shared header, helper C và binary ARM, hai .ko, Makefile, script và các log. Các ảnh terminal cùng ngày là minh chứng kèm theo; ảnh debug mã được ghi ngày 20/10/2026.

Tài liệu kỹ thuật

[S2] Linux Kernel 5.15. ioctl based interfaces.
https://www.kernel.org/doc/html/v5.15/driver-api/ioctl.html

[S3] Linux Kernel 5.15. Miscellaneous Devices và VFS.
https://www.kernel.org/doc/html/v5.15/driver-api/misc_devices.html
https://www.kernel.org/doc/html/v5.15/filesystems/vfs.html

[S4] Linux Kernel 5.15. Memory Management APIs.
https://www.kernel.org/doc/html/v5.15/core-api/mm-api.html

[S5] Linux man pages. delete_module(2).
https://man7.org/linux/man-pages/man2/delete_module.2.html

[S6] Linux Kernel 5.15. Overview of the Linux Virtual File System.
https://www.kernel.org/doc/html/v5.15/filesystems/vfs.html

[S7] Linux Kernel. Atomic types, tài liệu về ngữ nghĩa API atomic.
https://www.kernel.org/doc/html/v6.6/core-api/wrappers/atomic_t.html

[S8] Linux Kernel 5.15. UBI File System.
https://www.kernel.org/doc/html/v5.15/filesystems/ubifs.html

Tài liệu và các trang kỹ thuật được đối chiếu ngày 21/10/2026. Đề Assignment 01 How to Write a Device Driver quy định phạm vi chức năng, bố cục bài nộp và yêu cầu báo cáo.

Ghi nhận hỗ trợ: ChatGPT được dùng để hỗ trợ giải thích, rà soát và soạn bản tổng hợp từ mã, log, ảnh. Người học cần tự kiểm tra nội dung, hoàn thiện sơ đồ và bảo đảm cách sử dụng phù hợp quy định của học phần.